AutoFFT: A Template-Based FFT Code Auto-Generation Framework for ARM and x86 CPUs

Zhihao Li, Haipeng Jia, Yunquan Zhang, Tun Chen, Liang Yuan, Luning Cao, Xiao Wang
Institute of Computing Technology, Chinese Academy of Sciences

11/19/2019



Outline

- **Background**
- A template-based code generation framework
  - Optimized butterfly patterns
  - Code generation
- Performance
- Summary and future work

Introduction to the DFT

- The discrete Fourier transform (DFT) : transforms x(n) from time domain to frequency domain.

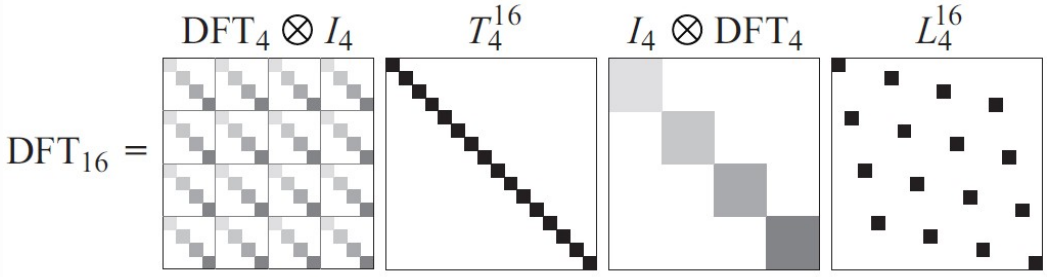
$$y_k = \sum_{j=0}^{N-1} \omega_n^{jk} \cdot x_j \; (0 \le k < n) \quad \omega_n = e^{-\frac{2\pi i}{n}} (i = \sqrt{-1})$$

- The DFT is the most important discrete transform, used to perform Fourier analysis in many scientific computing applications.
- The DFT also can be expressed as the multiplication of DFT matrix and the input sequence.

$$DFT_n = \begin{bmatrix} 1 & 1 & 1 & & 1 \\ 1 & \omega_n^1 & \omega_n^2 & \cdots & \omega_n^{n-1} \\ 1 & \omega_n^2 & \omega_n^4 & & \omega_n^{2(n-1)} \\ & \vdots & & \ddots & \vdots \\ 1 & \omega_n^{n-1} & \omega_n^{2(n-1)} & \cdots & \omega_n^{(n-1)(n-1)} \end{bmatrix}, y = DFT_n \cdot x$$

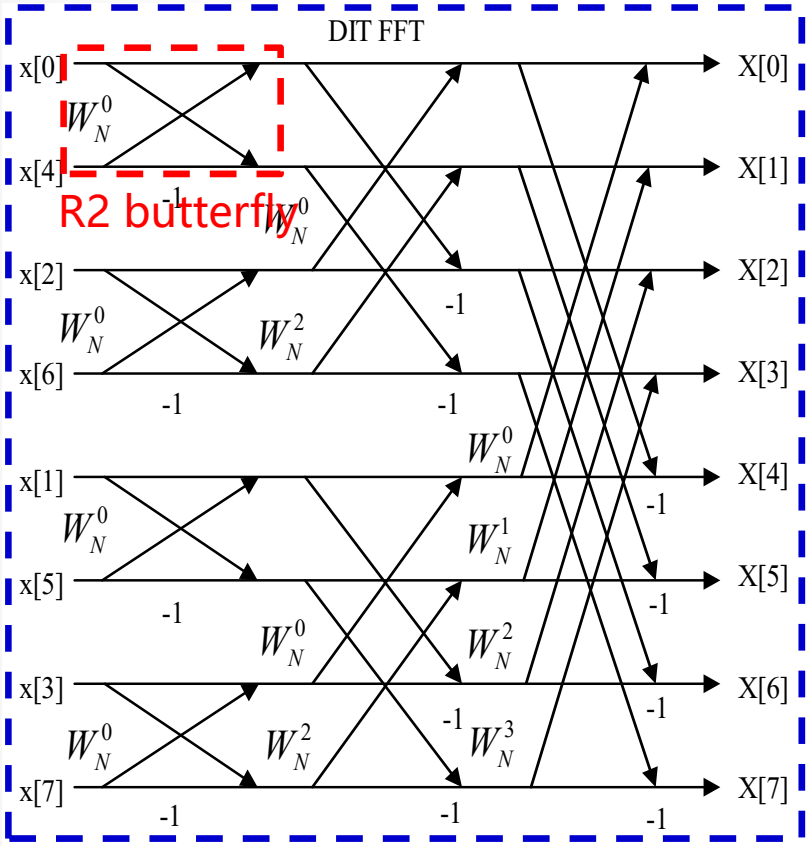
Introduction to the FFT

- The fast Fourier transform (FFT)
  - a fast algorithm for computing the DFT of a sequence.
- The Cooley-Tukey algorithm is the most popular FFT algorithm
  - adopts a divide-and-conquer approach to recursively break down a large DFT into smaller DFTs;
  - reduces the computational complexity of the DFT from O(n^2) to O(nlogn).



The Cooley-Tukey algorithm

Network



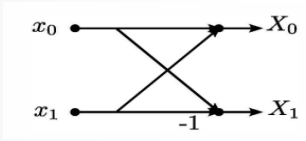
Algorithm: Cooley-Turkey algorithm
IN: input array
OUT: output array
W: twiddles array

```
for(i <- 0; i < stage_num; i++)//stage
    for(j <- 0; i < section_num; j++)//section
        for(k <- 0; i < butterfly_num; k++)//butterfly
          IN[i,j,k] = IN[i,j,k] * W[i, k]
          calButterfly(OUT[i,j,k], IN[i,j,k])
        endfor
    endfor
endfor
```

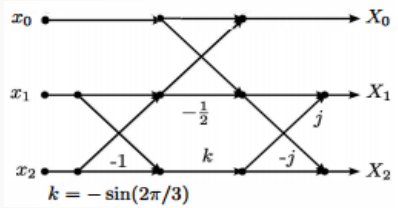
Problems in butterfly optimizations

- Problems in butterfly implementations and optimizations
  - Many types of butterflies (Each radix corresponds to one type of butterfly)
  - Each butterfly contains its own calculation path
  - Butterflies with larger radices mean more complicated paths

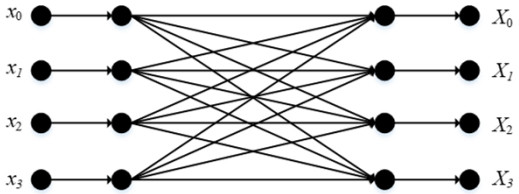
R2



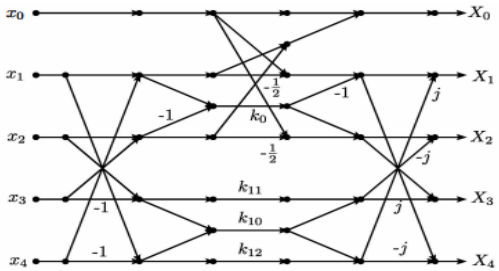
R3



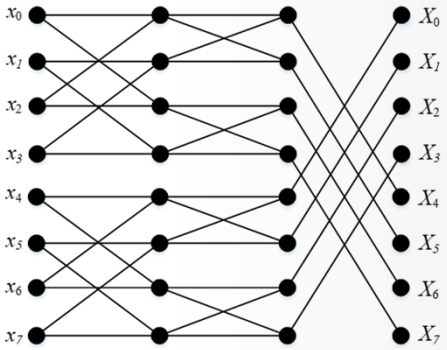
R4



R5



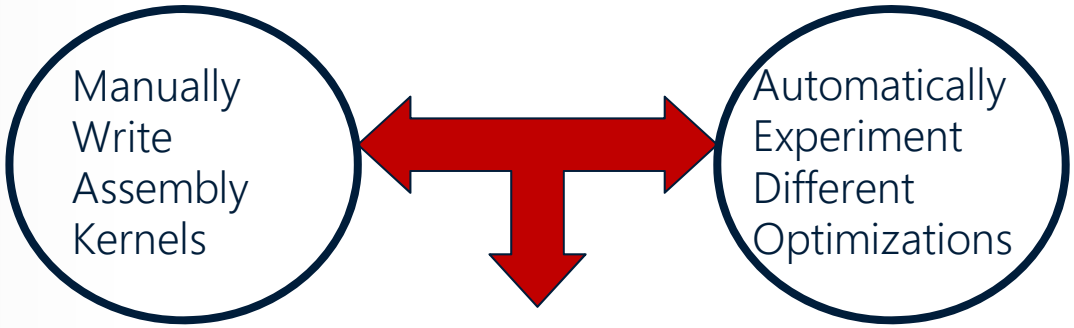
R8



Two ways to obtain highly optimized FFT code

- Manual implementation
  - Write assembly code to achieve peak performance for varying CPUs
    - Require enough knowledge of algorithm and architecture
    - Repetitive implementations for varying CPUs
    - Bind tightly with the features of architectures
    - Can not port onto new machine without performance degradation
- Auto-tuning
  - Automatically experiment different optimization strategies and select the optimal result for target CPU
    - Generate codes in high-level languages
    - Rely on the performance of compilers
    - Unsatisfactory performance

Two ways to generate highly optimized FFT code



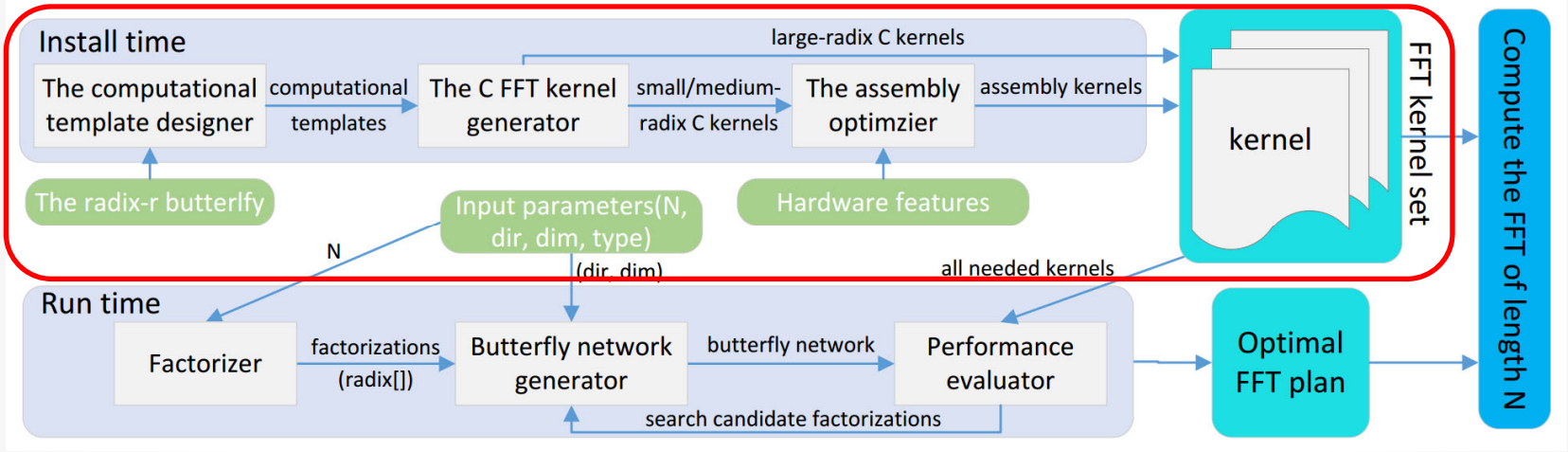


Is there a way to auto-generate all kinds of butterfly kernels, and at the same time achieve high performance?

Outline

- Background
- **A template-based code generation framework**
  - Optimized butterfly patterns
  - Code generation
- Performance
- Summary and future work

The code generation framework



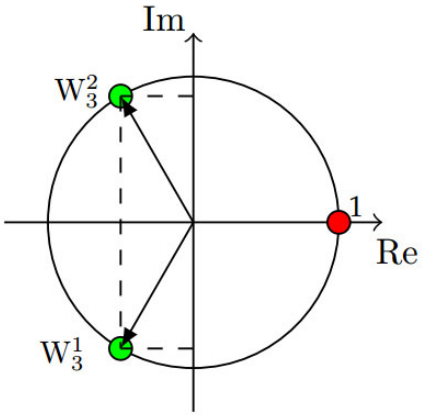
- Install time: generates high-performance FFT kernels of all kinds of radices for varying CPUs.
- Runtime: According to the size of input, transform type, and hardware features, search the optimal butterfly network.

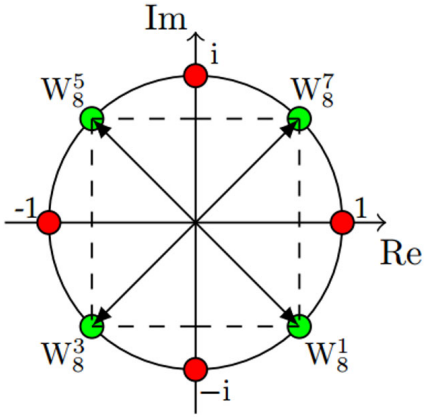
Optimized butterfly patterns

- The essence of radix-r butterfly is DFT_r:

$$Y_k = \sum_{i=0}^{r} x_i * W_r^{ik}$$

- To utilize symmetry and periodicity of twiddles, split r into two cases:
  - prime numbers
  - powers of two





$$a_{ik} = \mathrm{Re}(W_r^{ik})$$
$$b_{ik} = Im(W_r^{ik})$$

Optimized butterfly patterns

- **The pair pattern** (prime radices)
  - Symmetry of W_r^{ik} and $W_r^{(r-i)k}$ on the horizontal direction

Y_0	W_5^0	W_5^0	W_5^0	W_5^0	W_5^0
Y_1	W_5^0	W_5^1	W_5^2	W_5^3	W_5^4
Y_2	W_5^0	W_5^2	W_5^4	W_5^1	W_5^3
Y_3	W_5^0	W_5^3	W_5^1	W_5^4	W_5^2
Y_4	W_5^0	W_5^4	W_5^3	W_5^2	W_5^1



Y_0	W_5^0	W_5^0	W_5^0	W_5^0	W_5^0
Y_1	W_5^0	a_1+b_1j	a_2+b_2j	a_2-b_2j	a_1-b_1j
Y_2	W_5^0	W_5^2	W_5^4	W_5^1	W_5^3
Y_3	W_5^0	W_5^3	W_5^1	W_5^4	W_5^2
Y_4	W_5^0	W_5^4	W_5^3	W_5^2	W_5^1

Vertical

Horizontal

  - Combine like terms

$$Y_k = \sum_{i=0}^{r-1} x_i W_r^{ik} = x_0 + \sum_{i=1}^{r-1} x_i W_r^{ik}$$

$$= x_0 + \sum_{i=1}^{m} (x_i + x_{r-i}) a_{ik} + \sum_{i=1}^{m} (x_i - x_{r-i}) b_{ik} \cdot j$$

Optimized butterfly patterns

- **The pair pattern** (prime radices)
  - Similarly, for Y_{r-k}, we have:

Y_0	W_5^0	W_5^0	W_5^0	W_5^0	W_5^0
Y_1	W_5^0	W_5^1	W_5^2	W_5^3	W_5^4
Y_2	W_5^0	W_5^2	W_5^4	W_5^1	W_5^3
Y_3	W_5^0	W_5^3	W_5^1	W_5^4	W_5^2
Y_4	W_5^0	W_5^4	W_5^3	W_5^2	W_5^1



Y_0	W_5^0	W_5^0	W_5^0	W_5^0	W_5^0
Y_1	W_5^0	W_5^1	W_5^2	W_5^3	W_5^4
Y_2	W_5^0	W_5^2	W_5^4	W_5^1	W_5^3
Y_3	W_5^0	W_5^3	W_5^1	W_5^4	W_5^2
Y_4	W_5^0	a_1-b_1j	a_2-b_2j	a_2+b_2j	a_1+b_1j

Vertical

Horizontal

$$Y_{r-k} = \sum_{i=0}^{r-1} x_i W_r^{i(r-k)} = x_0 + \sum_{i=1}^{r-1} x_i W_r^{i(r-k)}$$

$$= x_0 + \sum_{i=1}^{r-1} x_i W_r^{ir-ik} = x_0 + \sum_{i=1}^{r-1} x_i W_r^{-ik}$$

$$= x_0 + \sum_{i=1}^{m} (x_i + x_{r-i}) a_{ik} - \sum_{i=1}^{m} (x_i - x_{r-i}) b_{ik} \cdot j$$

Optimized butterfly patterns

- **The pair pattern** (prime radices)
  - Symmetry on the vertical direction

$$Y_k = \sum_{i=0}^{r-1} x_i W_r^{ik} = x_0 + \sum_{i=1}^{r-1} x_i W_r^{ik}$$

$$= x_0 + \sum_{i=1}^{m} (x_i + x_{r-i}) a_{ik} + \sum_{i=1}^{m} (x_i - x_{r-i}) b_{ik} \cdot j$$

$$Y_{r-k} = \sum_{i=0}^{r-1} x_i W_r^{i(r-k)} = x_0 + \sum_{i=1}^{r-1} x_i W_r^{i(r-k)}$$

$$= x_0 + \sum_{i=1}^{r-1} x_i W_r^{ir-ik} = x_0 + \sum_{i=1}^{r-1} x_i W_r^{-ik}$$

$$= x_0 + \sum_{i=1}^{m} (x_i + x_{r-i}) a_{ik} - \sum_{i=1}^{m} (x_i - x_{r-i}) b_{ik} \cdot j$$

Y_0	W_5^0	W_5^0	W_5^0	W_5^0	W_5^0
Y_1	W_5^0	W_5^1	W_5^2	W_5^3	W_5^4
Y_2	W_5^0	W_5^2	W_5^4	W_5^1	W_5^3
Y_3	W_5^0	W_5^3	W_5^1	W_5^4	W_5^2
Y_4	W_5^0	W_5^4	W_5^3	W_5^2	W_5^1

Vertical

Horizontal

Optimized butterfly patterns

- **The pair pattern** (prime radices)
  - Symmetry on the vertical direction
    - Define P_k and Q_k,

$$P_k = \sum_{i=1}^{m}(x_i + x_{r-i})a_{ik}$$
$$Q_k = \sum_{i=1}^{m}(x_i - x_{r-i})b_{ik} \cdot j$$

    - then we have:

$$Y_k = x_0 + P_k + Q_k$$

    - Y_k and Y_{r-k} are calculated together

$$Y_{r-k} = x_0 + P_k - Q_k$$

Y_0	W_5^0	W_5^0	W_5^0	W_5^0	W_5^0
Y_1	W_5^0	W_5^1	W_5^2	W_5^3	W_5^4
Y_2	W_5^0	W_5^2	W_5^4	W_5^1	W_5^3
Y_3	W_5^0	W_5^3	W_5^1	W_5^4	W_5^2
Y_4	W_5^0	W_5^4	W_5^3	W_5^2	W_5^1

Horizontal

Vertical

Optimized butterfly patterns

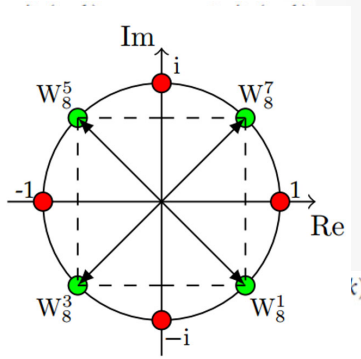
- **The quad pattern** (power-of-two radices)
  - Twiddles W_r^{ik}、$W_r^{(r-i)k}$、$W_r^{(\frac{r}{2}-i)k}$、$W_r^{(\frac{r}{2}+i)k}$ have better symmetries, then $Y_k, Y_{r-k}, Y_{\frac{r}{2}-k}$, and $Y_{\frac{r}{2}+k}$ are calculated together using the similar manner.

$$Y_k = x_0 + (-1)^k x_{r/2} + x_{r/4} W_r^{r/4 \cdot k} + x_{3r/4} W_r^{3r/4 \cdot k}$$
$$+ \sum_{i=1}^{m} \left((x_i + x_{r-i}) + (-1)^k (x_{r/2-i} + x_{r/2+i}) \right) a_{ik}$$
$$+ \sum_{i=1}^{m} \left((x_i - x_{r-i}) - (-1)^k (x_{r/2-i} - x_{r/2+i}) \right) b_{ik} \cdot j$$

$$Y_{r-k} = x_0 + (-1)^k x_{r/2} + x_{r/4} W_r^{r/4 \cdot k} + x_{3r/4} W_r^{3r/4 \cdot k}$$
$$+ \sum_{i=1}^{m} \left((x_i + x_{r-i}) + (-1)^k (x_{r/2-i} + x_{r/2+i}) \right) a_{ik}$$
$$- \sum_{i=1}^{m} \left((x_i - x_{r-i}) - (-1)^k (x_{r/2-i} - x_{r/2+i}) \right) b_{ik} \cdot j$$

$$Y_{r/2-k} = x_0 + (-1)^k x_{r/2} + x_{r/4} W_r^{r/4 \cdot (r/2-k)} + x_{3r/4} W_r^{3r/4 \cdot (r/2-k)}$$
$$+ \sum_{i=1}^{m} (-1)^i \left((x_i + x_{r-i}) + (-1)^k (x_{r/2-i} + x_{r/2+i}) \right) a_{ik}$$
$$- \sum_{i=1}^{m} (-1)^i \left((x_i - x_{r-i}) - (-1)^k (x_{r/2-i} - x_{r/2+i}) \right) b_{ik} \cdot j$$

$$Y_{r/2+k} = x_0 + (-1)^k x_{r/2} + x_{r/}$$
$$+ \sum_{i=1}^{m} (-1)^i \left((x_i + x_{r-i}) + (-1)^{} (x_{r/2-i} + x_{r/2+i}) \right) a_{ik}$$
$$+ \sum_{i=1}^{m} (-1)^i \left((x_i - x_{r-i}) - (-1)^k (x_{r/2-i} - x_{r/2+i}) \right) b_{ik} \cdot j$$



Optimized butterfly patterns

- **The quad pattern** (power-of-two radices)
  - Twiddles W_r^{ik}、$W_r^{(r-i)k}$、$W_r^{\left(\frac{r}{2}-i\right)k}$、$W_r^{(\frac{r}{2}+i)k}$ have better symmetries, then $Y_k, Y_{r-k}, Y_{\frac{r}{2}-k}$, and $Y_{\frac{r}{2}+k}$ are calculated together using the similar manner.
  - Twiddles contain better periodicities when gcd(r, k)>1. Note that gcd() indicates the greatest common divisor.)

gcd(8,k)	Y_0	W_8^0	W_8^0	W_8^0	W_8^0	W_8^0	W_8^0	W_8^0	W_8^0
1	Y_1	W_8^0	W_8^1	W_8^2	W_8^3	W_8^4	W_8^5	W_8^6	W_8^7
2	Y_2	W_8^0	W_8^2	W_8^4	W_8^6	W_8^0	W_8^2	W_8^4	W_8^6
1	Y_3	W_8^0	W_8^3	W_8^6	W_8^1	W_8^4	W_8^7	W_8^2	W_8^5
4	Y_4	W_8^0	W_8^4	W_8^0	W_8^4	W_8^0	W_8^4	W_8^0	W_8^4
1	Y_5	W_8^0	W_8^5	W_8^2	W_8^7	W_8^4	W_8^1	W_8^6	W_8^3
2	Y_6	W_8^0	W_8^6	W_8^4	W_8^2	W_8^0	W_8^6	W_8^4	W_8^2
1	Y_7	W_8^0	W_8^7	W_8^6	W_8^5	W_8^4	W_8^3	W_8^2	W_8^1

Optimized butterfly patterns

- **The quad pattern** (power-of-two radices)
  - Twiddles W_r^{ik}、$W_r^{(r-i)k}$、$W_r^{\left(\frac{r}{2}-i\right)k}$、$W_r^{(\frac{r}{2}+i)k}$ have better symmetries, then $Y_k, Y_{r-k}, Y_{\frac{r}{2}-k}$, and $Y_{\frac{r}{2}+k}$ are calculated together using the similar manner.
  - Twiddles contain better periodicities when gcd(r, k)>1. Note that gcd() indicates the greatest common divisor.)
  - To obtain Y_k: combine the same terms first, then apply the symmetric properties. ($Y_{r-k}, Y_{\frac{r}{2}-k}$, and $Y_{\frac{r}{2}+k}$ are calculated simultaneously.)

$$Y_k = \sum_{i=0}^{r-1} x_i W_r^{ik} = \sum_{i=0}^{r-1} x_i W_r^{(ik \bmod r)}$$

$$= \sum_{s=0}^{p-1}\sum_{t=0}^{q-1} x_{t+s\cdot q} W_r^{tk} = \sum_{t=0}^{q-1}\left(\sum_{s=0}^{p-1} x_{t+s\cdot q}\right)\cdot W_r^{tk}$$

$$\hat{x}_{(t,k)} = \sum_{s=0}^{p-1} x_{t+s\cdot q}$$

$$Y_k = \sum_{t=0}^{q-1} \hat{x}_{(t,k)} \cdot W_r^{tk}$$

Code generation

- **The computational templates**
  - Extract the templates based on the pair and quad patterns
  - Formalize modular expression of butterflies to enable generation of high-level kernels

Algorithm 1 butterfly_kernel(out, in, tw, r, i, isFirst)

Input: in[]: the input data of current stage; tw[]: twiddles; r: radix; i: the ith butterfly; isFirst: whether it is the first stage.
Output: out[]: output data.

```
1:  Load inputs into tmpIn[] from in[] according to i
2:  if isFirst=0 then
3:      Load twiddles into tmpTW[] from tw[]
4:      for i←1 to r do
5:          CPX_MUL(tmpIn[i], tmpIn[i], tmpTW[i-1])
6:      end for
7:  end if
8:  CALC_LIKE_TERMS(add, sub, tmpIn, r)
9:  CALC_OUT_SPECIAL(tmpOut, add, sub, r)
10: if r is a prime number then
11:     for i←1 to r/2 do
12:         CALC_OUT_PAIR(tmpOut, add, sub, i, r)
13:     end for
14: else if r is a power of two then
15:     for i←1 to r/4 do
16:         CALC_OUT_QUAD(tmpOut, tmpIn, i, r)
17:     end for
18: end if
19: Store outputs from tmpOut[] to out[]
```

```
CPX_ADD(out,in1,in2):
out.r=in1.r+in2.r
out.i=in1.i+in2.i

CPX_SUB(out,in1,in2):
out.r=in1.r-in2.r
out.i=in1.i-in2.i

CPX_MLA(out,in1,in2,s):
out.r=in1.r+s*in2.r
out.i=in1.i+s*in2.i

CPX_MUL_S(out,in,s):
out.r=s*in.r
out.i=s*in.i

CPX_OUT(head,tail,A,B):
head.r=A.r-B.i
head.i=A.i+B.r
tail.r=A.r+B.i
tail.i=A.i-B.r

CPX_MUL(out,in1,in2):
rr=in1.r*in2.r
ii=in1.i*in2.i
ri=in1.r*in2.i
ir=in1.i*in2.r
out.r=rr-ii
out.i=ri+ir
```

```
CALC_LIKE_TERMS(add[],sub[],in[],N):
m=(N-1)/2, t=N-1;
if(N%2==0)
CPX_ADD(add[0],in[0],in[N/2]);
CPX_SUB(sub[0],in[0],in[N/2]);
else
add[0]=in[0];sub[0]=-in[0];
endif
for(h=1; i<=m; h++,t--)
CPX_ADD(add[h],in[h],in[t]);
CPX_SUB(sub[h],in[h],in[t]);
end for

CALC_OUT0(out[],add[],sub[],N):
m=N/2,out[0]=out[m]=0
for(i=0;i<=m;i++)
CPX_ADD(out[0],out[0],add[i]);
end for
if(N%2==0)
for(i=0;i<=m;i+=2)
CPX_ADD(out[m],out[m],add[i]);
CPX_SUB(out[m],out[m],add[i+1]);
end for
end if

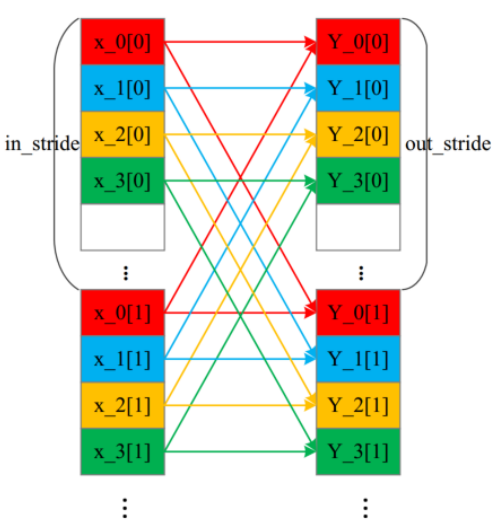
CALC_OUT_PAIR(out[],add[],sub[],k,N):
CPX_MLA(P,add[0],add[1], r);
CPX_MUL_S(Q,sub[1], i);
for(i=2;i<=N/2;++i)
CPX_MLA(P,P,add[i], r);
CPX_MLA(Q,Q,sub[i], i);
end for
CPX_OUT(out[k] ,out[N-k] ,P,Q);

CALC_OUT_QUAD(out[],add[],sub[],k,N):
initial Z,E,F,G,H as (0,0);
q = log(k/g);
a = N / (2^q);
for(j=c=0; j < power(2,q); ++j, c=a*j)
A = compute_A(add,sub,a,c);
CPX_ADD(Z,Z,A);
D = compute_D(add,sub,a,c);
CPX_ADD(G,G,D);
CPX_SUB(H,H,D);
end for
s = (q == 0) ? 1 : power(2,q);
for(i=1; i < N/4; ++i)
for(j=c=0; j < s; ++j, c=a*j)
B = compute_B(add,sub,a,c,i);
CPX_ADD(E,E,B);
C = compute_C(add,sub,a,c,i);
CPX_ADD(G,G,C);
if(i%2==1)
CPX_SUB(F,F,B);
CPX_ADD(H,H,C);
else
CPX_ADD(F,F,B);
CPX_SUB(H,H,C);
end if
end for
end for
CPX_ADD(TMP,Z,E);
CPX_OUT(out[k] , out[N-k] , TMP, G);
CPX_ADD(TMP,Z,F);
CPX_OUT(out[N/2-k] , out[N/2+k] , TMP, H);
```

Code generation

- **The optimization templates**
  - Butterfly vectorization
  - Instruction mapping rules betwee high-level templates and optimiz templates
  - Register allocation strategies
    - Small and medium radices (assembly kernels)
    - Large radices (C kernels)

Meta Templates	ARMv8 Optimization Templates	x86-64 Optimization Templates
CPX_ADD(out,in1,in2):	**CPX_ADD**(out_r,out_i,in1_r,in1_i,in2_r,in2_i):	**CPX_ADD**(out,in1,in2): vaddps %in1, %in2, %out



Serial version

load x_0[0]
load x_0[1]
calculate 1 butterfly
store Y_0[0]
store Y_0[1]

load x_1[0]
load x_1[1]
calculate 1 butterfly
store Y_1[0]
store Y_1[1]

load x_2[0]
load x_2[1]
calculate 1 butterfly
store Y_2[0]
store Y_2[1]

load x_3[0]
load x_3[1]
calculate 1 butterfly
store Y_3[0]
store Y_3[1]

vectorize

SIMD version

vload v4_x0 (x_0[0],x_1[0],x_2[0],x_3[0])
vload v4_x1 (x_0[1],x_1[1],x_2[1],x_3[1])
calculate 4 butterflies
vstore v4_X0 (Y_0[0],Y_1[0],Y_2[0],Y_3[0])
vstore v4_X1 (Y_0[1],Y_1[1],Y_2[1],Y_3[1])

CPX_OUT(head,tail,A,B): head.r=A.r-B.i head.i=A.i+B.r tail.r=A.r+B.i tail.i=A.i-B.r	**CPX_OUT**(h_r,h_i,t_r,t_i,A_r,A_i,B_r,B_i): fsub h_r.4s, A_r.4s, B_i.4s fadd h_i.4s, A_i.4s, B_r.4s fadd t_r.4s, A_r.4s, B_i.4s fsub t_i.4s, A_i.4s, B_r.4s	**CPX_OUT**(head,tail,A,B): vshufps 0xb1, %B, %B, %tail vaddsubps %tail, %A, %head vfmsubadd231ps ONE(%rip), %A, %tail

Outline

- Background
- A template-based code generation framework
  - Optimized butterfly patterns
  - Code generation
- **Performance**
- Summary and future work

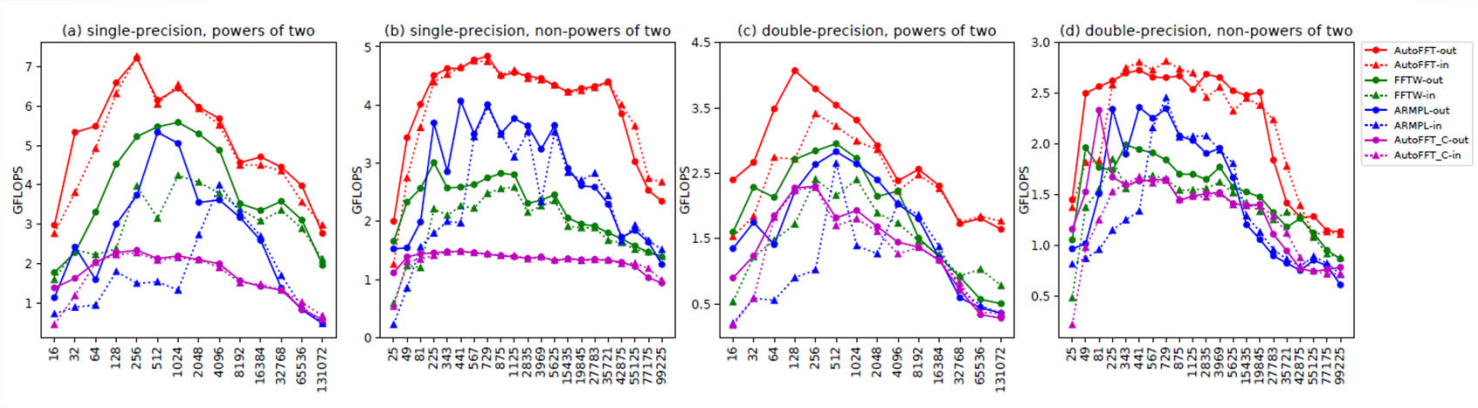
Performance

- Experimental Environment

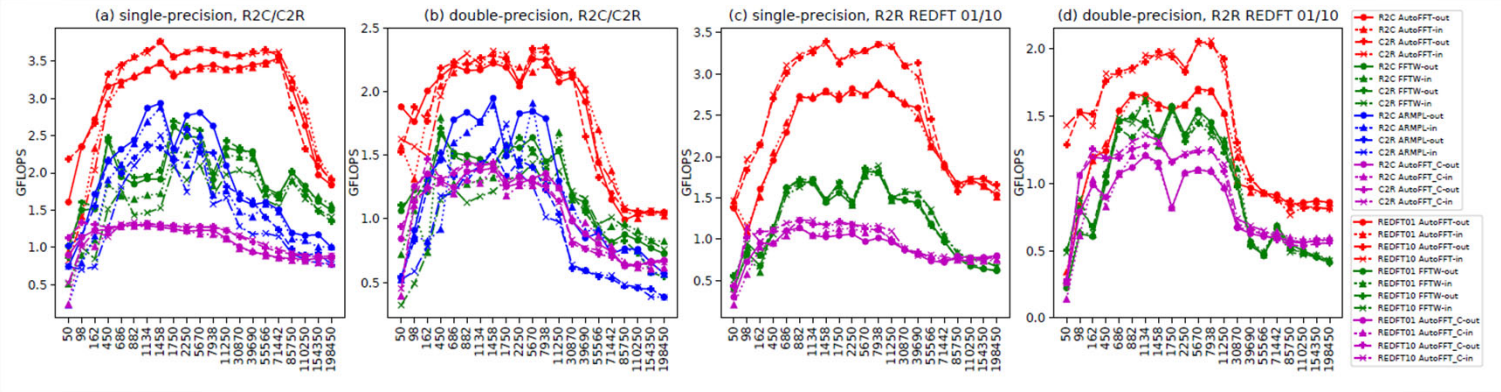
CPU	ARMv8	Xeon E7-4850 v3
Arch.	AArch64	Haswell
Freq.	2.2 GHz	2.2 GHz
SIMD	128	256
L1 cache	32 KB	32 KB
GCC	4.9.3	5.5.0
FFTW	3.3.8	3.3.8
ARMPL	19.2.0	-
Intel MKL	-	2019 Update 4

- Libraries compared
  - FFTW, ARMPL, MKL, and AutoFFT_C (C kernels)
- The two-part string figure legend (e.g.: AutoFFT-out)
  - The first part indicates the FFT library (AutoFFT/FFTW/ARMPL/AutoFFT_C)
  - The second part indicates out-of-place / in-place transform.

Performance

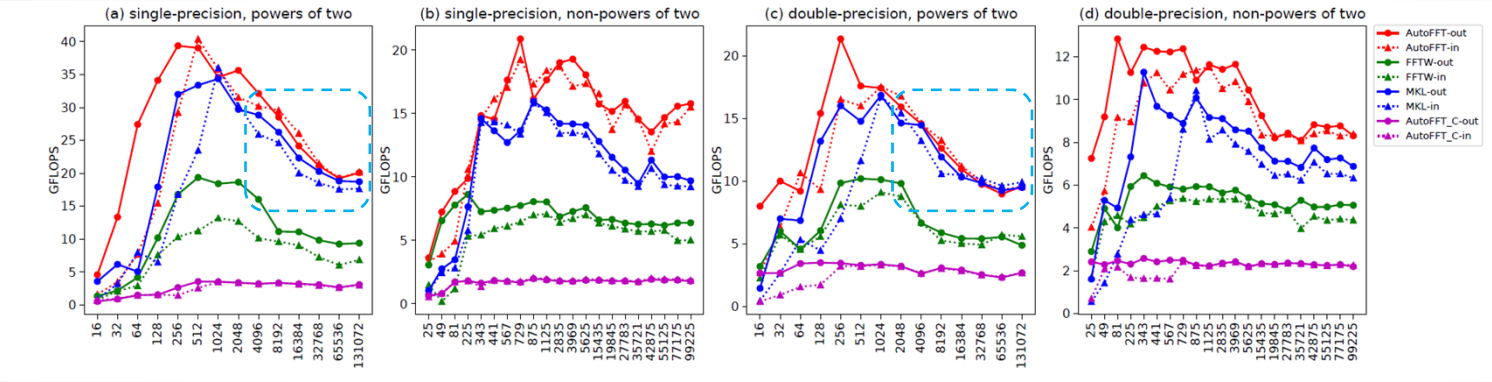


The Performance of 1D C2C (complex-to-complex) FFTs on ARM

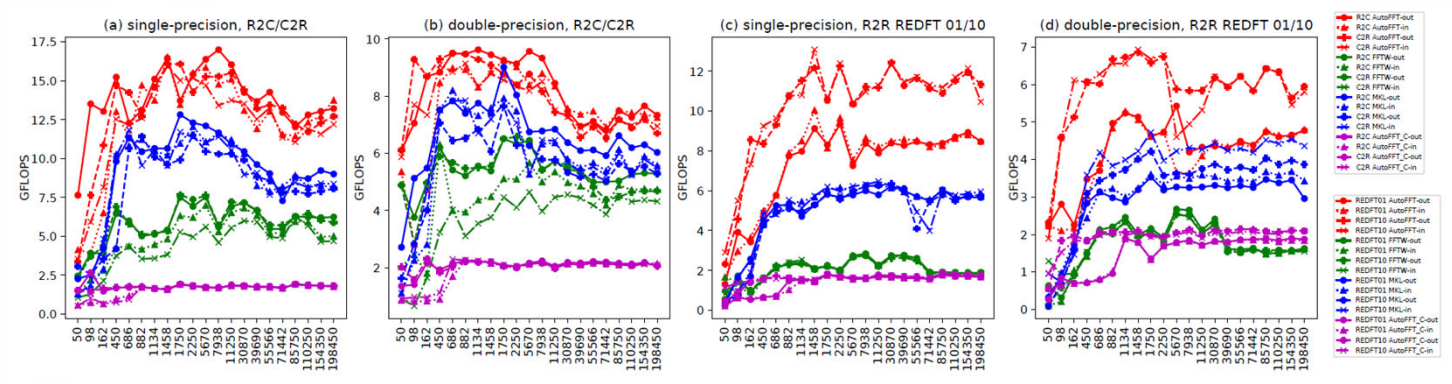


The Performance of 1D Real FFTs on ARM

Performance



The Performance of 1D C2C FFTs on Intel



The Performance of 1D Real FFTs on Intel

Outline

- Background
- A template-based code generation framework
  - Optimized butterfly patterns
  - Code generation
- Performance
- **Summary and future work**

Summary and future work

- Summary
  - Propose two optimized butterfly patterns for prime numbers and powers of two (the pair and quad patterns)
  - Propose a template-based framework to generate high-performance butterfly kernels
- Future work
  - Generalize the optimized patterns to all natural numbers
  - Extend AutoFFT to many-core platforms